

XAI Project

Heart Failure Clinical Records

Confronto tra modelli interpretabili basati su regole
e modelli ensemble non interpretabili

Mattia Manneschi

Laboratorio di Explainable AI

Contents

1	Introduzione del problema e dataset	2
1.1	Contesto e motivazione	2
1.2	Il Dataset	2
1.3	EDA — Distribuzioni per classe	3
1.4	EDA — Matrice di correlazione	4
2	Metodologia	4
2.1	Struttura del Progetto	4
2.2	Pipeline di Elaborazione dei Dati	5
2.3	Modelli Interpretabili Basati su Regole	5
2.4	Modelli Ensemble Non Interpretabili	6
2.5	Selezione degli Iperparametri	6
3	Risultati	10
3.1	Metriche Comparative	10
3.2	Curve ROC	11
3.3	Matrici di Confusione	12
3.4	Regole Complete: tutti i modelli interpretabili	14
3.5	Cross-Validation 10-fold	17
3.6	RF, GRL e BRL senza feature <code>time</code>	18
3.7	Analisi delle Feature	19
3.8	Partial Dependence Plots e ICE	21
4	Conclusioni	23

1 Introduzione del problema e dataset

1.1 Contesto e motivazione

Il problema. L'insufficienza cardiaca è una condizione clinica grave in cui il cuore non riesce a pompare sangue in modo sufficiente. Identificare i fattori di rischio associati al decesso è fondamentale per supportare le decisioni cliniche e migliorare la prognosi dei pazienti.

Obiettivo. Il progetto confronta due famiglie di modelli di machine learning per la predizione della mortalità: modelli interpretabili basati su regole (rule-based) e modelli non interpretabili basati su ensemble di alberi. L'obiettivo è valutare il trade-off tra performance predittiva e interpretabilità in un contesto clinico ad alta criticità.

Explainable AI in ambito clinico. In medicina, un modello non spiegabile — per quanto accurato — non è direttamente utilizzabile come strumento di supporto decisionale: il medico deve poter comprendere e validare le motivazioni di ogni predizione. I modelli intrinsecamente interpretabili soddisfano questo requisito senza richiedere metodi post-hoc aggiuntivi (es. SHAP, LIME), che possono introdurre approssimazioni e spiegazioni fuorvianti.

Struttura del report. Il Capitolo 1 introduce il problema clinico, descrive il dataset e presenta l'analisi esplorativa. Il Capitolo 2 descrive la metodologia: pipeline di elaborazione dei dati, modelli utilizzati e selezione degli iperparametri. Il Capitolo 3 presenta i risultati: metriche comparative, curve ROC, matrici di confusione, regole apprese, cross-validation, esperimento di ablazione della feature `time`, analisi delle feature (importance globale) e Partial Dependence Plots con ICE. Il Capitolo 4 raccoglie le conclusioni e le considerazioni cliniche.

1.2 Il Dataset

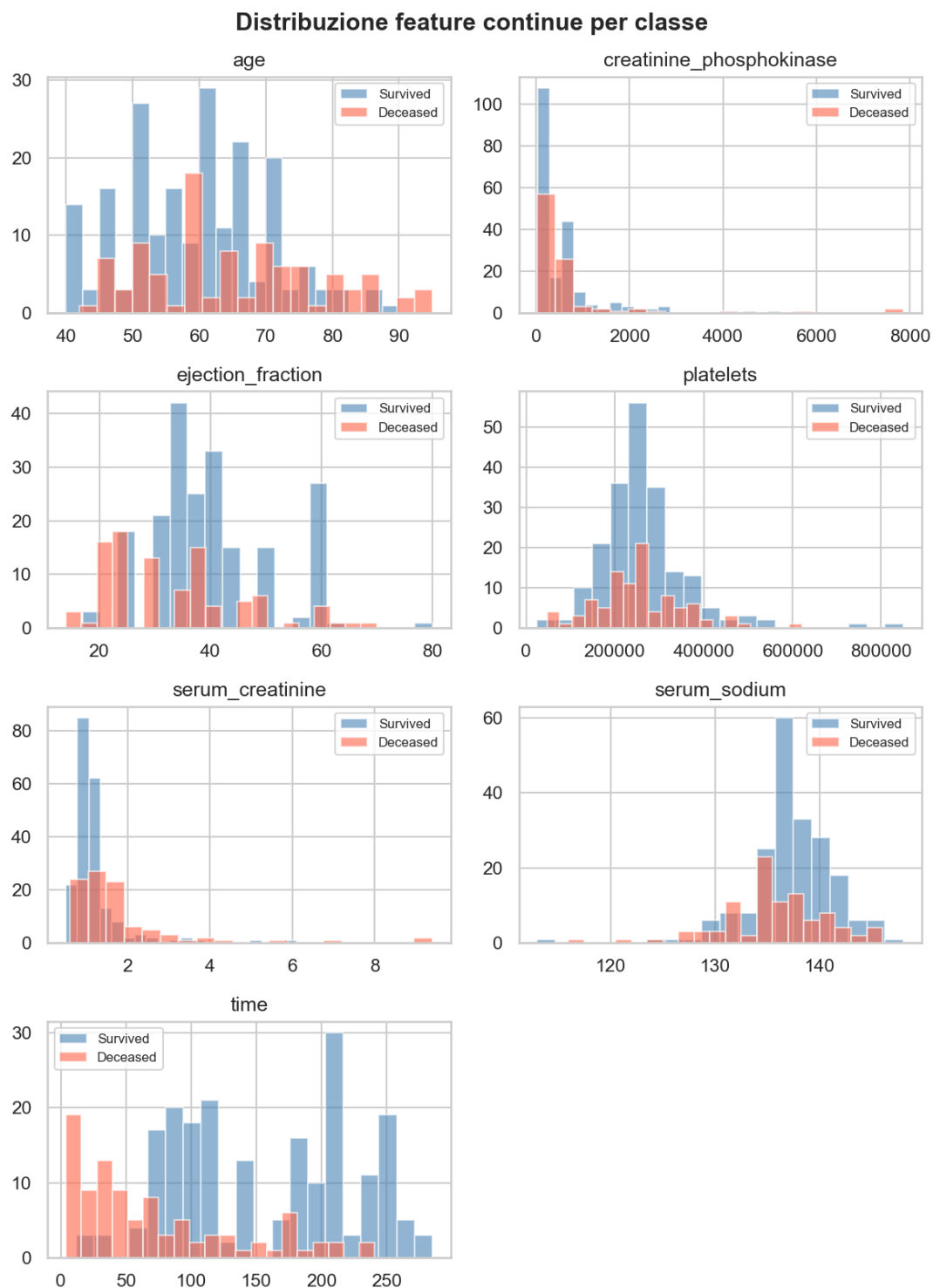
Il dataset *Heart Failure Clinical Records* (UCI, id=519) raccoglie cartelle cliniche di 299 pazienti con insufficienza cardiaca seguiti in follow-up. La variabile target `DEATH_EVENT` indica se il paziente è deceduto durante il periodo di osservazione.

Campioni: 299 | Decessi: 96 (32.1%) | Sopravvissuti: 203 (67.9%)

Gestione delle feature: binarie vs continue. Le 12 feature si dividono in 5 binarie (`anaemia`, `diabetes`, `high_blood_pressure`, `sex`, `smoking`) e 7 continue. I modelli ensemble e la maggior parte dei rule-based le ricevono direttamente. RuleFit richiede standardizzazione (`StandardScaler`), mentre `BayesianRuleList` richiede feature esclusivamente binarie: le variabili continue vengono discretizzate in bin tramite `KBinsDiscretizer` (one-hot, 4 bin quantili) prima del fitting.

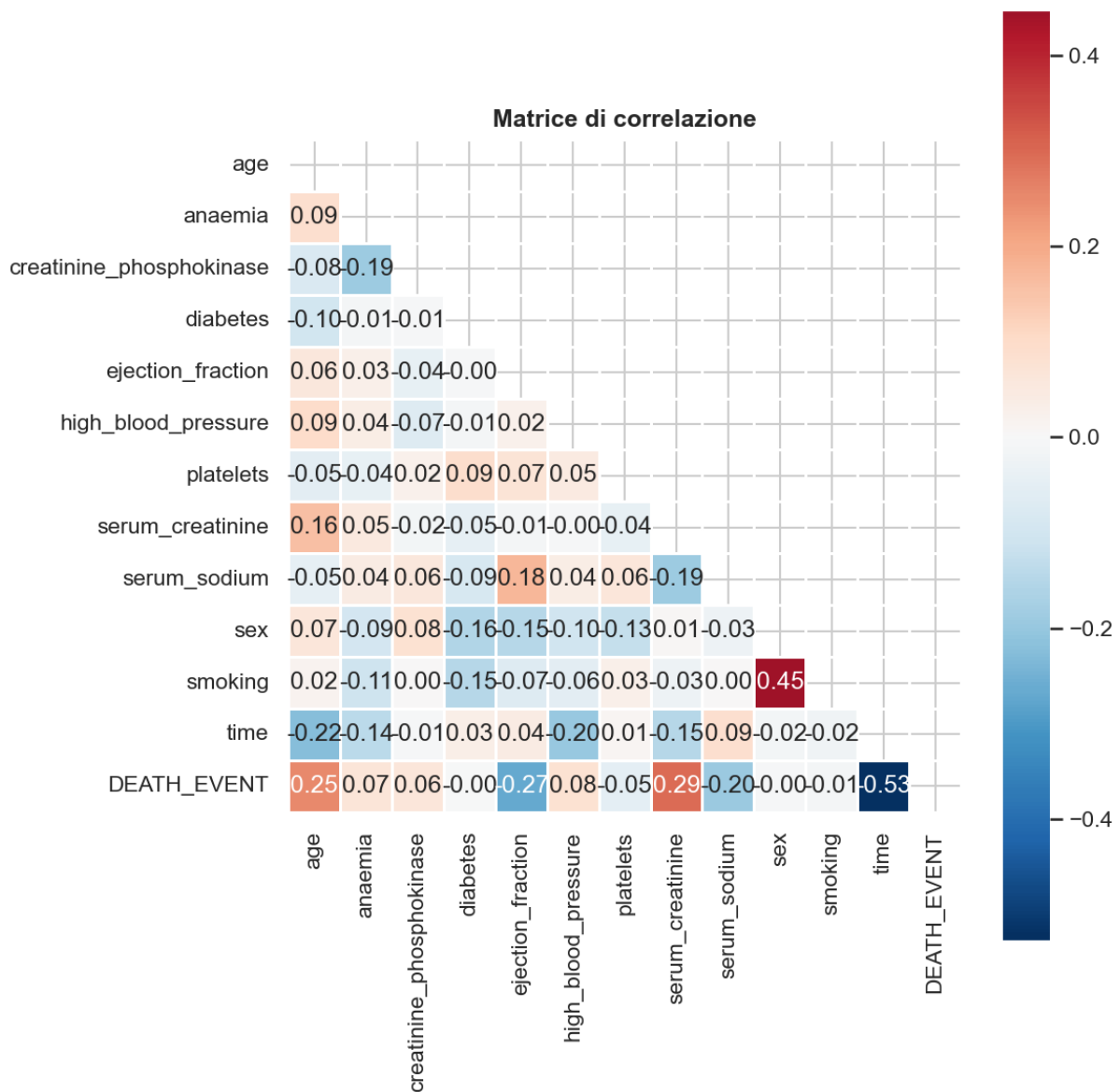
Feature	Tipo	Descrizione
age	continua	Eta' del paziente (anni)
anaemia	binaria	Presenza di anemia (0/1)
creatinine_phosphokinase	continua	Livello CPK nel sangue (mcg/L)
diabetes	binaria	Presenza di diabete (0/1)
ejection_fraction	continua	Percentuale di sangue pompata dal ventricolo sinistro (%)
high_blood_pressure	binaria	Presenza di ipertensione (0/1)
platelets	continua	Conta piastrinica (kiloplatelets/mL)
serum_creatinine	continua	Livello di creatinina serica (mg/dL)
serum_sodium	continua	Livello di sodio sierico (mEq/L)
sex	binaria	Sesso biologico (0=F, 1=M)
smoking	binaria	Fumatore (0/1)
time	continua	Durata del follow-up (giorni)

1.3 EDA — Distribuzioni per classe



Ogni istogramma sovrappone la distribuzione di una feature per sopravvissuti (blu) e deceduti (rosso). **time** è il discriminatore più potente: un follow-up breve corrisponde quasi sempre a decesso. **serum_creatinine** alta indica compromissione renale; **ejection_fraction** bassa segnala ridotta capacità di pompa. **platelets** e **creatinine_phosphokinase** mostrano distribuzioni largamente sovrapposte.

1.4 EDA — Matrice di correlazione



`time` ha la correlazione più forte con `DEATH_EVENT` (-0.53): pazienti con follow-up lungo tendono a sopravvivere. `serum_creatinine` ($+0.29$) ed `ejection_fraction` (-0.27) sono i segnali biochimici più informativi. Le feature binarie mostrano correlazioni deboli ($< \pm 0.15$).

2 Metodologia

2.1 Struttura del Progetto

Il progetto è organizzato in una cartella `src/` con sei moduli distinti, ciascuno con una responsabilità precisa. Questa separazione permette di modificare una componente senza toccare le altre e rende il codice più leggibile e testabile.

Modulo	Responsabilita'
config.py	Centralizza iperparametri, path e costanti
data_loader.py	Download UCI, cache CSV, split stratificato 60/20/20, tre varianti di preprocessing
models.py	Factory function per ciascuno dei 9 modelli; accetta <code>**params</code> per override
evaluate.py	Calcolo metriche, grafici, cross-validation e grid search sul validation set (ROC-AUC)
main.py	Orchestrazione dell'intera pipeline end-to-end
report_latex.py	Legge gli artefatti salvati e genera il report LaTeX/PDF

Output della pipeline. Ogni esecuzione di `main.py` produce artefatti su disco: i dati grezzi vengono cachati in `data/`; le metriche e le regole vengono salvate in `results/files/`; i grafici in `results/plots/`; il report finale in `Report/`.

Dipendenze principali. `scikit-learn` gestisce preprocessing, metriche e cross-validation. `imodels` fornisce le implementazioni dei modelli rule-based (`RuleFit`, `GreedyRuleList`, `BayesianRuleList`, `FIGS`, `SkopeRules`). `wittgenstein` implementa `RIPPER`. `xgboost` implementa `XGBoost`. `ucimlrepo` scarica il dataset direttamente dal repository UCI.

2.2 Pipeline di Elaborazione dei Dati

Split stratificato 60/20/20. Il dataset viene diviso in tre parti con doppio `train_test_split` stratificato: training (≈ 179 campioni, 60%), validation (≈ 60 campioni, 20%) e test (≈ 60 campioni, 20%). La stratificazione garantisce che ogni partizione mantenga la stessa proporzione di deceduti ($\sim 32\%$) presente nel dataset originale. Il validation set viene usato esclusivamente per la selezione degli iperparametri; il test set è mantenuto completamente separato e usato una sola volta per la valutazione finale.

Tre varianti di preprocessing. I modelli richiedono rappresentazioni diverse: (1) *raw* — il `DataFrame` originale con le 12 feature, usato da `GreedyRuleList`, `FIGS`, `SkopeRules`, `RIPPER` e dagli ensemble; (2) *scalata* — `StandardScaler` porta le feature continue a media 0 e varianza 1, necessaria per `RuleFit` che usa Lasso internamente; (3) *discretizzata* — `KBinsDiscretizer` (4 bin quantili, one-hot) produce ≈ 33 colonne binarie, obbligatoria per `BayesianRuleList`.

PreparedData — struttura dati condivisa. Il preprocessing è incapsulato in un data-class `PreparedData` con i campi: `X_train/X_val/X_test` (raw), le varianti scalate e discretizzate per ciascuna partizione, `X_trainval/y_trainval` (train+val, usato per la cross-validation), `y_train/y_val/y_test` e `scale_pos_weight` per `XGBoost`. `Scaler` e discretizzatore sono fittati solo sul training set e applicati con `transform()` a validation e test, evitando data leakage.

Sbilanciamento delle classi. Il dataset contiene 203 sopravvissuti e 96 deceduti ($\sim 32\%$ positivi). `Random Forest` usa `class_weight='balanced'`; `XGBoost` usa `scale_pos_weight` calcolato come rapporto negativi/positivi nel training set.

2.3 Modelli Interpretabili Basati su Regole

I modelli basati su regole producono predizioni if-then direttamente leggibili da un esperto di dominio, senza richiedere strumenti post-hoc aggiuntivi.

RuleFit. Estrae regole da un ensemble di alberi e le pondera con regressione Lasso. Il coefficiente di ogni regola indica il suo contributo alla predizione; richiede feature standardizzate.

GreedyRuleList (GRL). Costruisce una lista ordinata di regole if-then in modo greedy. Ogni regola è un decision stump (una condizione su una feature); il parametro `max_depth` controlla la lunghezza della lista. Si applica la prima regola soddisfatta; se nessuna si attiva, vale la regola di default. Importante: GRL produce sempre regole a condizione singola per design della libreria `imodels`.

BayesianRuleList (BRL). Apprende una lista di regole tramite MCMC su feature binarie. Le variabili continue vengono discretizzate in bin prima del fitting; produce stime probabilistiche con IC al 95%. Il parametro `maxcardinality` controlla il numero massimo di condizioni per regola; dal tuning sul validation set è risultato ottimale il valore 3.

FIGS — Fast Interpretable Greedy-Tree Sums. Somma di piccoli alberi decisionali, ciascuno appreso greedy sui residui del precedente. La predizione è la somma dei valori “Val” raggiunti traversando ogni albero; per i classificatori viene poi applicata una softmax. Struttura additiva trasparente con capacità espressiva maggiore di un singolo albero.

SkopeRules. Genera regole ad alta precisione tramite sub-campionamenti ripetuti di ensemble di alberi, poi de-duplica quelle simili. La profondità degli alberi base (`max_depth`) controlla il numero di condizioni AND per regola; ogni regola è corredata da precisione e recall.

RIPPER — Repeated Incremental Pruning to Produce Error Reduction. Apprende regole congiuntive (AND tra più condizioni simultanee) in modo incrementale, potando ogni regola per ridurre l'errore. La prima regola soddisfatta predice Decesso; se nessuna si attiva, la predizione è Sopravvissuto (DEFAULT). A differenza di GRL, può combinare più feature in un singolo antecedente.

2.4 Modelli Ensemble Non Interpretabili

I modelli ensemble combinano centinaia di alberi decisionali per ottenere performance superiori, ma il processo decisionale risulta opaco (black box). Sono usati come baseline di riferimento per valutare il gap di performance rispetto ai modelli interpretabili.

Random Forest. Allena N alberi su sotto-campioni casuali del dataset (bagging) e su sottoinsiemi casuali delle feature (feature bagging). La predizione finale è la media delle probabilità dei singoli alberi. Configurato con `class_weight='balanced'`.

Gradient Boosting (sklearn). Allena alberi sequenzialmente: ogni albero corregge gli errori del precedente minimizzando il gradiente della loss. Riduce il bias rispetto al bagging.

XGBoost. Implementazione ottimizzata del gradient boosting con regolarizzazione L1/L2, gestione nativa dei valori mancanti e parallelismo. Usa `scale_pos_weight` per bilanciare le classi.

2.5 Selezione degli Iperparametri

Il dataset (299 campioni) è suddiviso in tre parti stratificate: train 60%, validation 20%, test 20% ($\approx 179/60/60$ campioni). Gli iperparametri di GreedyRuleList, BayesianRuleList, SkopeRules e dei tre ensemble vengono scelti massimizzando il ROC-AUC sul validation set tramite grid search esaustiva. RuleFit, FIGS e RIPPER usano i parametri di default della libreria. La valutazione finale avviene sul test set, mantenuto separato durante tutto il processo di selezione.

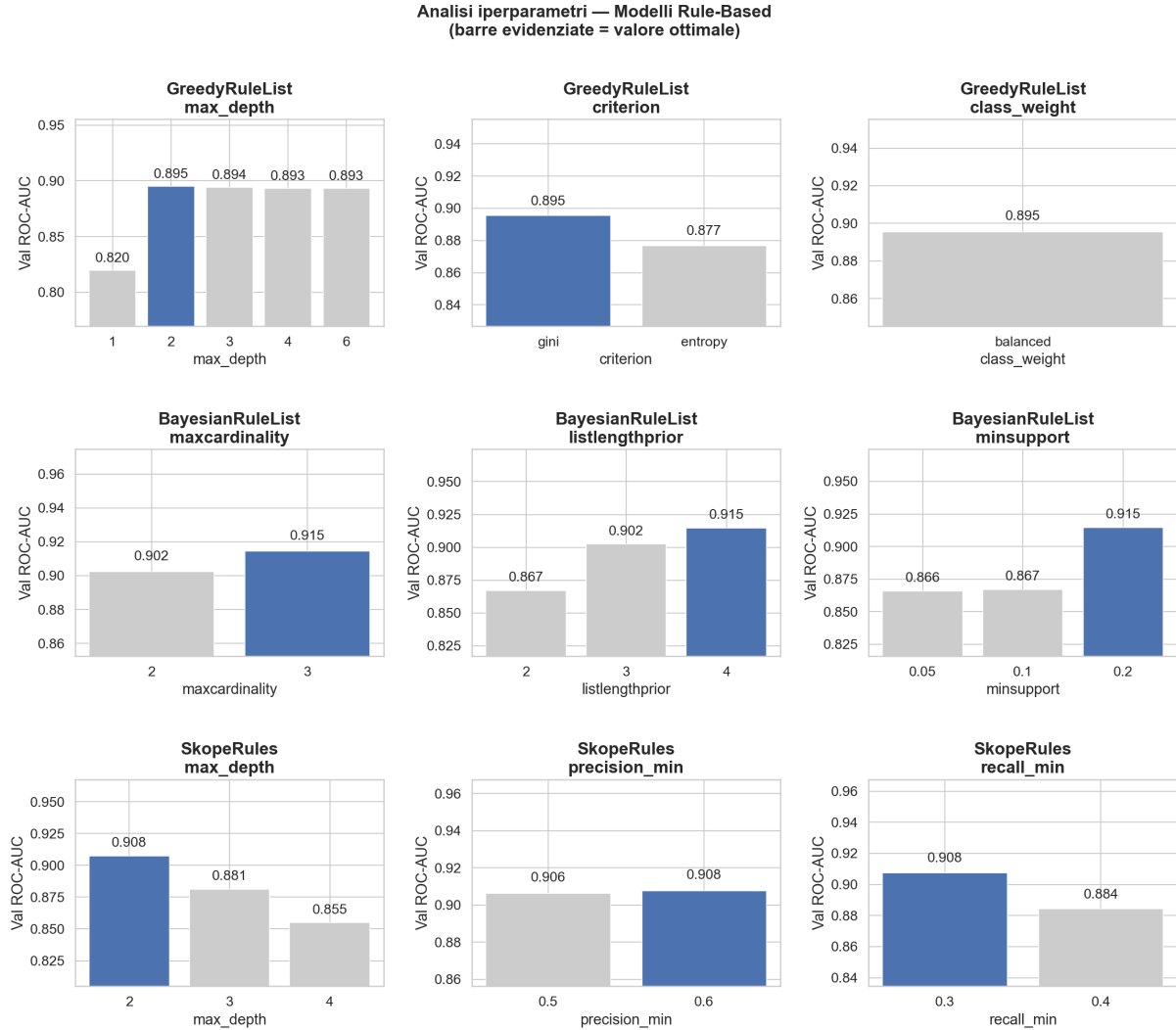
Grid di ricerca — migliori valori sul validation set

Modello	Parametro	Valori esplorati	Ottimo (val)
GreedyRuleList	max_depth	1, 2, 3, 4, 6	2
GreedyRuleList	criterion	gini, entropy	gini
GreedyRuleList	class_weight	None, balanced	None
BayesianRuleList	maxcardinality	2, 3	3
BayesianRuleList	listlengthprior	2, 3, 4	4
BayesianRuleList	minsupport	0.05, 0.1, 0.2	0.2
SkopeRules	max_depth	2, 3, 4	2
SkopeRules	precision_min	0.5, 0.6	0.6
SkopeRules	recall_min	0.3, 0.4	0.3
RandomForest	n_estimators	100, 200	100
RandomForest	max_depth	None, 5, 10	None
RandomForest	min_samples_leaf	1, 2, 5	5
GradientBoosting	n_estimators	100, 200	100
GradientBoosting	learning_rate	0.05, 0.1	0.1
GradientBoosting	max_depth	3, 4, 5	3
XGBoost	n_estimators	100, 200	100
XGBoost	learning_rate	0.05, 0.1	0.05
XGBoost	max_depth	3, 4, 5	4

Parametri fissi (non ottimizzati)

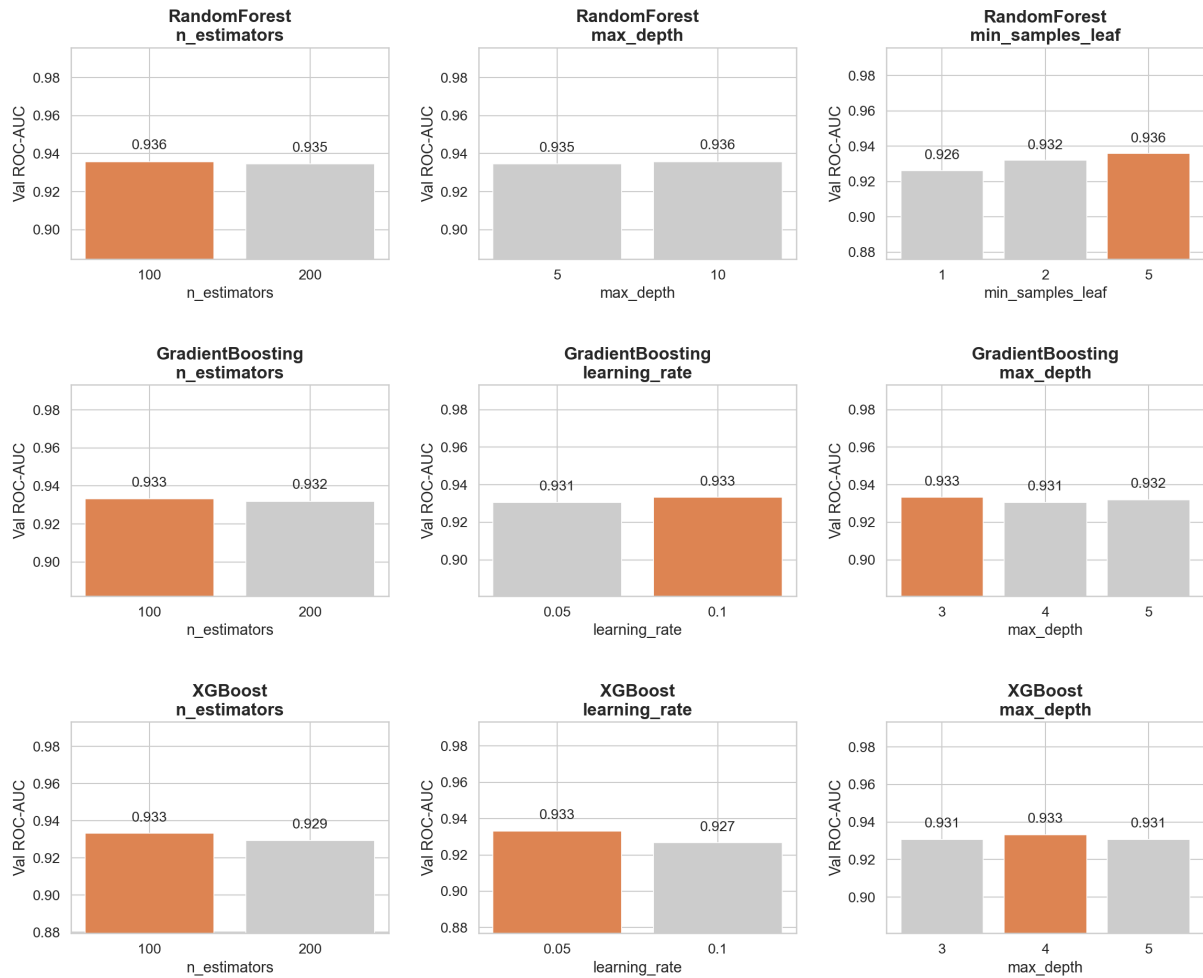
Parametro	Valore	Motivazione
RANDOM_STATE	42	Seed fisso per riproducibilit�
TEST_SIZE	0.20	20% test (≈ 60 campioni), mantenuto separato durante il tuning
VAL_SIZE	0.20	20% validation (≈ 60 campioni) per la scelta degli iperparametri
CV_FOLDS	10	10-fold CV su trainval (≈ 240 campioni): stima robusta della stabilit�
BRL_MAX_ITER	20 000	Iterazioni MCMC per la convergenza (ridotto a 3 000 durante il tuning)
RULEFIT_MAX_RULES	30	Limita le regole candidate; troppe degenerano in black-box
FIGS_MAX_RULES	10	Alberi nella somma FIGS; oltre 10 si perde interpretabilit�

Sensitivity agli iperparametri. Per ciascun modello ottimizzato   riportato il ROC-AUC sul validation set al variare di ogni iperparametro (marginalizzando sugli altri: per ogni valore del parametro si considera il massimo AUC tra tutte le combinazioni che lo includono). Le barre evidenziate corrispondono al valore ottimale selezionato.



Modelli rule-based. Per GreedyRuleList `max_depth=2` è il punto di equilibrio: liste troppo corte non catturano abbastanza pattern; liste lunghe non portano benefici su questo dataset. Il criterio di split ottimale è `gini`; `class_weight=None` indica che il modello gestisce lo sbilanciamento delle classi tramite la struttura delle regole stesse, senza ponderazione esplicita. Per BayesianRuleList la cardinalità ottimale è 3: un valore più alto permette al MCMC di esplorare congiunzioni più complesse durante la ricerca, sebbene in pratica converga su regole a condizione singola (come discusso in sezione 3.4). `listlengthprior=4` favorisce liste più lunghe, adatte alla variabilità del dataset; `minsupport=0.2` richiede che ogni regola copra almeno il 20% dei campioni di training, riducendo regole troppo specifiche. Per SkopeRules, `max_depth=2` genera regole bi-condizionali ad alta precisione; profondità maggiori producono overfitting. `precision_min=0.6` e `recall_min=0.3` bilanciano qualità e copertura delle regole selezionate, limitando la proliferazione di regole ridondanti.

Analisi iperparametri — Modelli Ensemble
(barre evidenziate = valore ottimale)

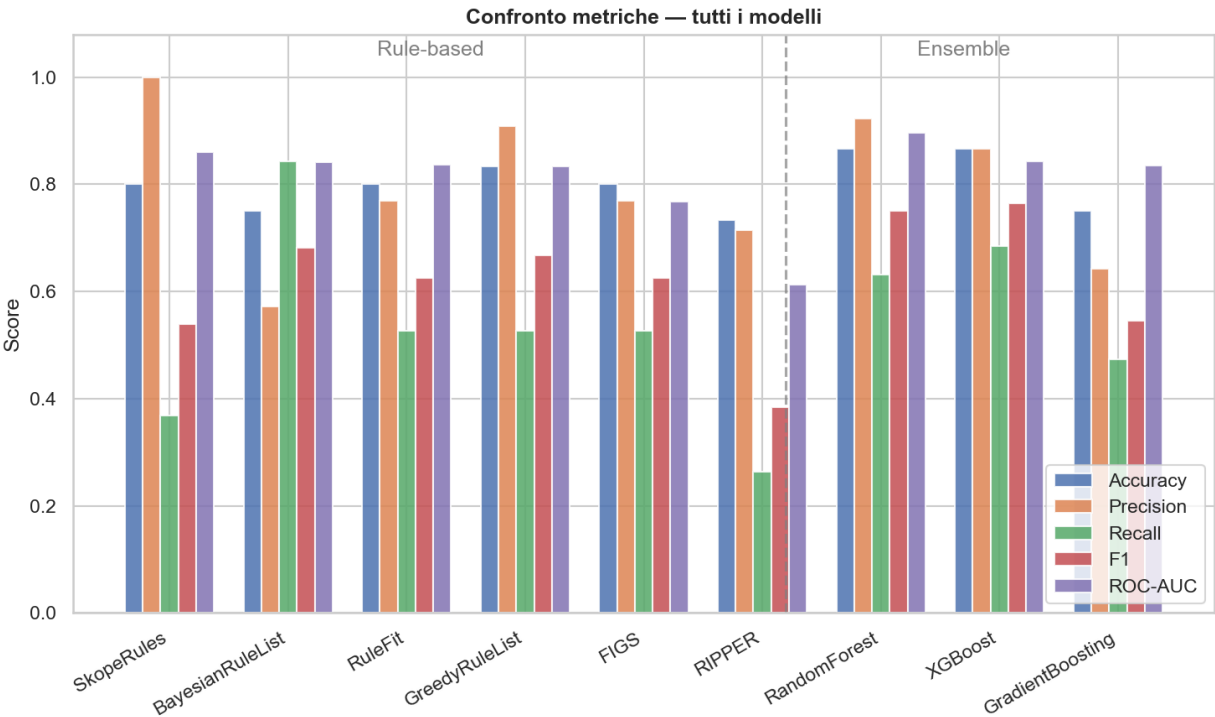


Modelli ensemble. Tutti e tre i modelli mostrano scarsa sensibilità al numero di stimatori nell'intervallo [100, 200]: la performance è già stabile a 100. Per Random Forest `max_depth=None` (alberi completi) domina; `min_samples_leaf=5` riduce la varianza su dataset piccoli. Gradient Boosting e XGBoost preferiscono alberi poco profondi (`max_depth=3-4`), coerente con la dimensione ridotta del dataset.

3 Risultati

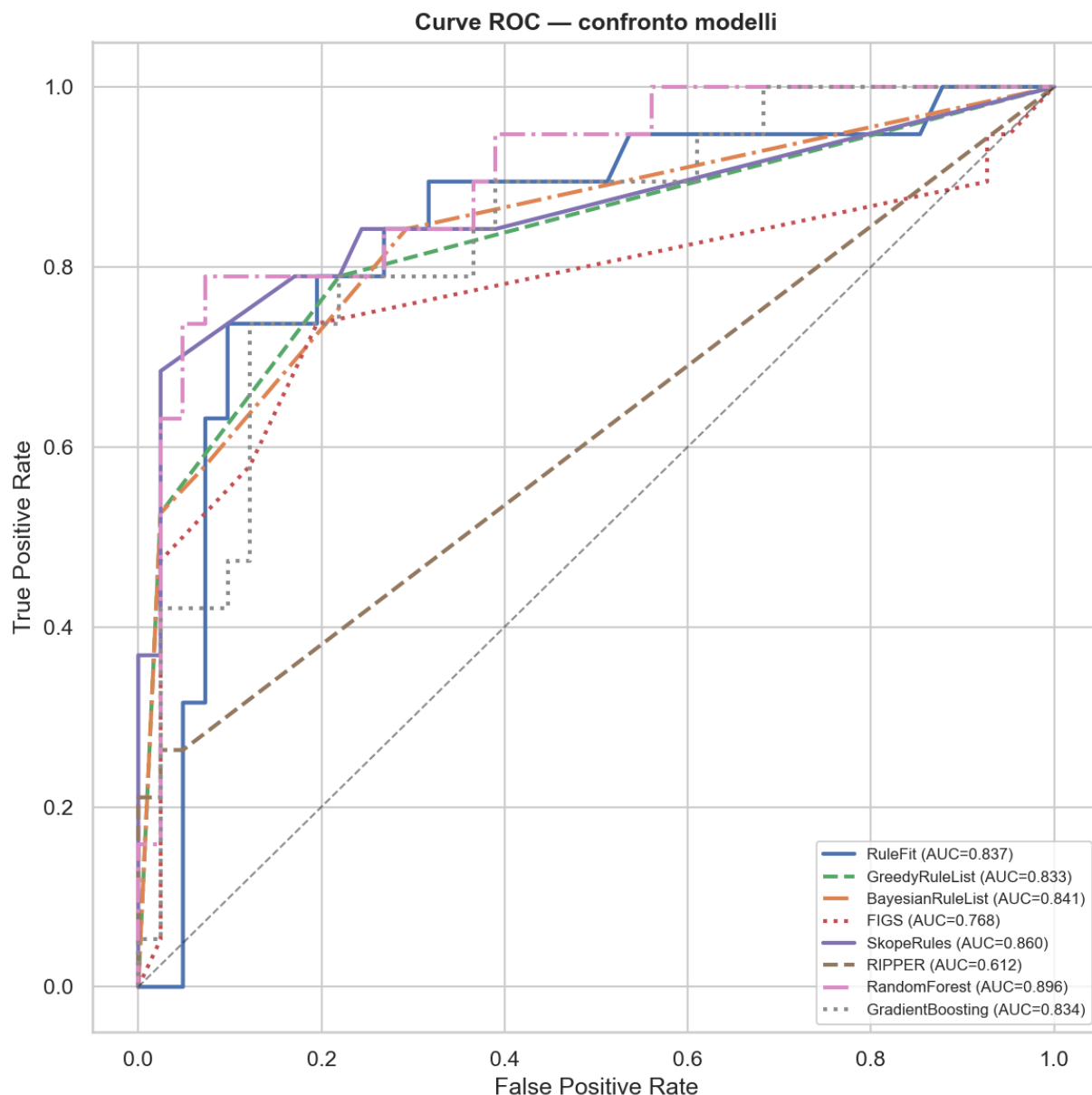
3.1 Metriche Comparative

Modello	Tipo	Accuracy	Precision	Recall	F1	ROC-AUC
RandomForest	Ensemble	0.8667	0.9231	0.6316	0.7500	0.8960
SkopeRules	Rule-based	0.8000	1.0000	0.3684	0.5385	0.8601
XGBoost	Ensemble	0.8667	0.8667	0.6842	0.7647	0.8434
BayesianRuleList	Rule-based	0.7500	0.5714	0.8421	0.6809	0.8408
RuleFit	Rule-based	0.8000	0.7692	0.5263	0.6250	0.8370
GradientBoosting	Ensemble	0.7500	0.6429	0.4737	0.5455	0.8344
GreedyRuleList	Rule-based	0.8333	0.9091	0.5263	0.6667	0.8331
FIGS	Rule-based	0.8000	0.7692	0.5263	0.6250	0.7677
RIPPER	Rule-based	0.7333	0.7143	0.2632	0.3846	0.6123



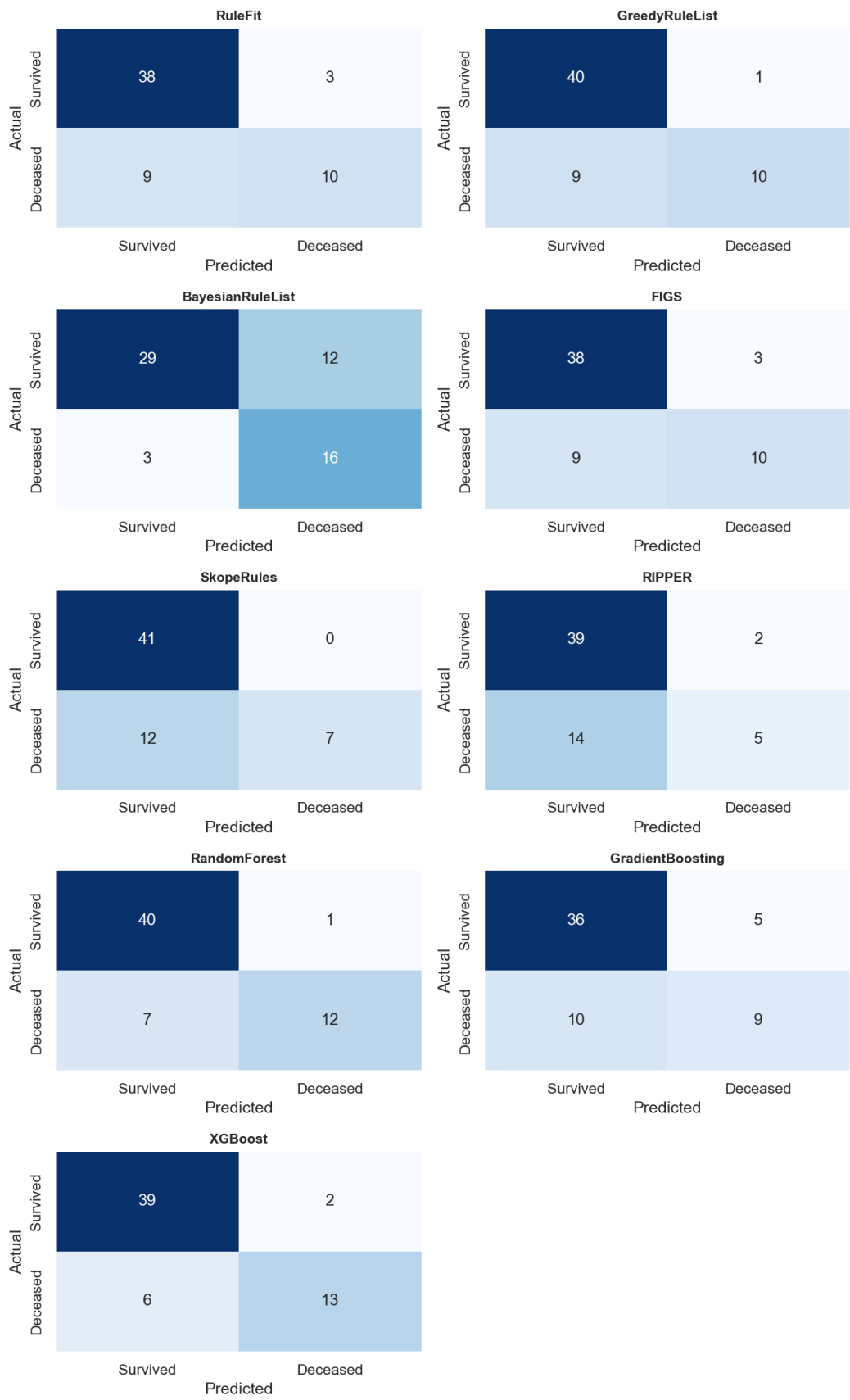
Accuracy e F1 riassumono la qualità complessiva, ma su classi sbilanciate possono essere fuorvianti. Precision misura quante predizioni “deceduto” sono corrette; Recall quante morti reali vengono identificate — in clinica un Recall basso è più pericoloso di una Precision bassa. ROC-AUC è la metrica più robusta: misura la capacità di separare le classi indipendentemente dalla soglia. GreedyRuleList raggiunge buon F1 e Accuracy con regole a condizione singola. SkopeRules ottiene Precision=1.0 con Recall molto basso: identifica pochi deceduti ma senza mai sbagliare. RIPPER produce regole congiuntive (AND) ma ha il ROC-AUC più basso tra tutti i modelli, dimostrando che la complessità strutturale non implica migliore capacità discriminativa.

3.2 Curve ROC



La curva ROC traccia il TPR (Recall) contro il FPR al variare della soglia; l'AUC riassume la performance in un unico numero indipendente dalla soglia (0.50 = casuale, 1.0 = perfetto). Random Forest raggiunge l'AUC più alto tra tutti i modelli. SkopeRules è il miglior rule-based in AUC (0.860), competitivo con i tre ensemble. RIPPER ha l'AUC più basso e anche F1 e Accuracy ridotti: la maggiore espressività strutturale delle sue regole congiuntive non si traduce in migliore capacità discriminativa su questo dataset.

3.3 Matrici di Confusione



Ogni cella indica: riga = classe reale, colonna = classe predetta. La diagonale raccoglie le predizioni corrette; gli elementi fuori diagonale sono gli errori. In clinica i due tipi di errore hanno costo asimmetrico: un falso negativo (morte non rilevata) è più grave di un falso positivo (falso allarme). `BayesianRuleList` massimizza il Recall sui deceduti — pochi falsi negativi — a scapito di più falsi positivi. Gli ensemble bilanciano meglio i due errori. `SkopeRules` ha Precision=1.0 ma Recall molto basso: identifica pochi deceduti, senza mai produrre falsi positivi.

3.4 Regole Complete: tutti i modelli interpretabili

GreedyRuleList. Lista IF/THEN sequenziale: la prima regola soddisfatta determina la classe, che non è necessariamente Decesso — ogni regola predice la classe di maggioranza del proprio segmento ($>50\%$ deceduti $\Rightarrow$ Decesso). GRL è strutturalmente limitato a una condizione per regola; il parametro `max_depth` controlla la lunghezza della lista.

Condizione	Precisione	Recall	Predizione	Campioni
<code>time <= 73.500</code>	0.830	0.672	Decesso	47
<code>serum_creatinine > 1.450</code>	0.500	0.091	Sopravvissuto	22
DEFAULT	0.927	0.843	Sopravvissuto	110

BayesianRuleList. Lista IF/ELSE con MCMC su feature discretizzate. Ogni antecedente ha al massimo `maxcardinality` condizioni congiuntive; il valore ottimale (3) è stato scelto sul validation set. $P(\text{decesso})$ è la probabilità posteriore con IC al 95%. Nonostante `maxcardinality=3` permetta antecedenti multi-feature, il MCMC converge su regole a condizione singola. Come confermato dall'esperimento in sezione 3.6 — dove BRL viene riaddestrato senza `time` — la preferenza per antecedenti unari è una proprietà intrinseca del MCMC su questo dataset: anche in assenza della feature dominante, le congiunzioni rimangono ridondanti dal punto di vista probabilistico.

Condizione	P(decesso)	IC 95%
<code>time ∈ (6.00, 71.50]</code>	80.9%	68.6%–90.6%
<code>ejection_fraction ∈ (15.00, 30.00]</code>	46.7%	29.4%–64.3%
<code>serum_creatinine ∈ (1.40, 6.80]</code>	27.8%	10.3%–49.9%
DEFAULT (nessuna regola attivata)	5.4%	1.8%–10.9%

RIPPER. Ogni regola ha un antecedente con più condizioni AND (tutte devono essere verificate simultaneamente). Se tutte le condizioni di una regola sono soddisfatte $\Rightarrow$ Decesso; nessuna regola soddisfatta $\Rightarrow$ Sopravvissuto (DEFAULT).

Regola	Predizione	Precisione	Recall	Campioni
<code>time <= 28.8</code>	Decesso	1.000	0.310	18
<code>ejection_fraction <= 25.0 AND high_blood_pressure = 1</code>	Decesso	0.769	0.172	13
<code>ejection_fraction <= 25.0 AND platelets ∈ (197200.0, 220400.0]</code>	Decesso	1.000	0.069	4
<code>time ∈ (28.8, 57.4] AND sex = 0</code>	Decesso	0.875	0.121	8
DEFAULT	Sopravvissuto	0.860	0.967	136

RuleFit — regole con coefficiente positivo (pro Decesso). Ogni regola è un percorso estratto da un ensemble di alberi; il coefficiente Lasso indica il peso nella predizione finale. Valori più alti indicano maggiore contributo al rischio di decesso. Le regole possono combinare più feature (AND implicito sul percorso).

Regola	Coefficiente
time <= 0.67896 and time > 0.23773	3.147
time <= -0.62527 and serum_creatinine > -0.44392	1.342
time > -0.69016 and creatinine_phosphokinase <= -0.01926 and creatinine_phosphokinase > -0.33791 and serum_creatinine > -0.14962	1.263
time <= 0.67896	1.130
time > -0.67718 and creatinine_phosphokinase > 0.25316 and ejection_fraction <= -0.41598 and serum_creatinine > -0.50523	1.006
time > -0.68367 and creatinine_phosphokinase <= -0.01926 and creatinine_phosphokinase > -0.49123 and serum_creatinine > 0.16921	0.877
time <= -0.67718 and serum_creatinine <= 0.35315 and serum_creatinine > -0.56654	0.704
time <= -1.02108 and creatinine_phosphokinase <= 1.22865	0.622

SkopeRules. Regole ad alta precisione estratte tramite sub-campionamenti di ensemble di alberi, poi de-duplicate. La profondità degli alberi base (`max_depth` ottimizzato sul validation set) controlla il numero di condizioni per regola. Ogni regola è corredata da Precisione e Recall sul training set.

Regola	Precisione	Recall
time <= 28.5	1.000	0.310
time <= 28.0	1.000	0.310
time <= 73.5 and serum_sodium <= 136.5	0.960	0.414
time <= 73.5 and high_blood_pressure > 0.5	0.955	0.362
time <= 52.0	0.909	0.517
time <= 67.5 and creatinine_phosphokinase > 142.0	0.893	0.431
time <= 67.5 and serum_creatinine > 0.75	0.881	0.638
time <= 73.5 and serum_creatinine > 0.95	0.878	0.621
time <= 66.5 and serum_creatinine > 0.75	0.878	0.621
time <= 73.5 and serum_creatinine > 0.85	0.867	0.672
time <= 55.0	0.861	0.534
time <= 67.5	0.860	0.638
time <= 73.5 and serum_creatinine > 0.75	0.848	0.672
time <= 78.5 and serum_creatinine > 0.85	0.837	0.707
time <= 78.5 and serum_creatinine > 0.75	0.820	0.707

(mostrate le prime 15 su 37 regole apprese, ordinate per precisione decrescente)

FIGS — Fast Interpretable Greedy-Tree Sums. FIGS è una somma di alberi: ogni albero produce una coppia di valori (uno per classe), le coppie vengono sommate elemento per elemento e una softmax sui due valori risultanti fornisce le probabilità finali; la classe con probabilità maggiore è la predizione. Di seguito la struttura completa degli alberi appresi:

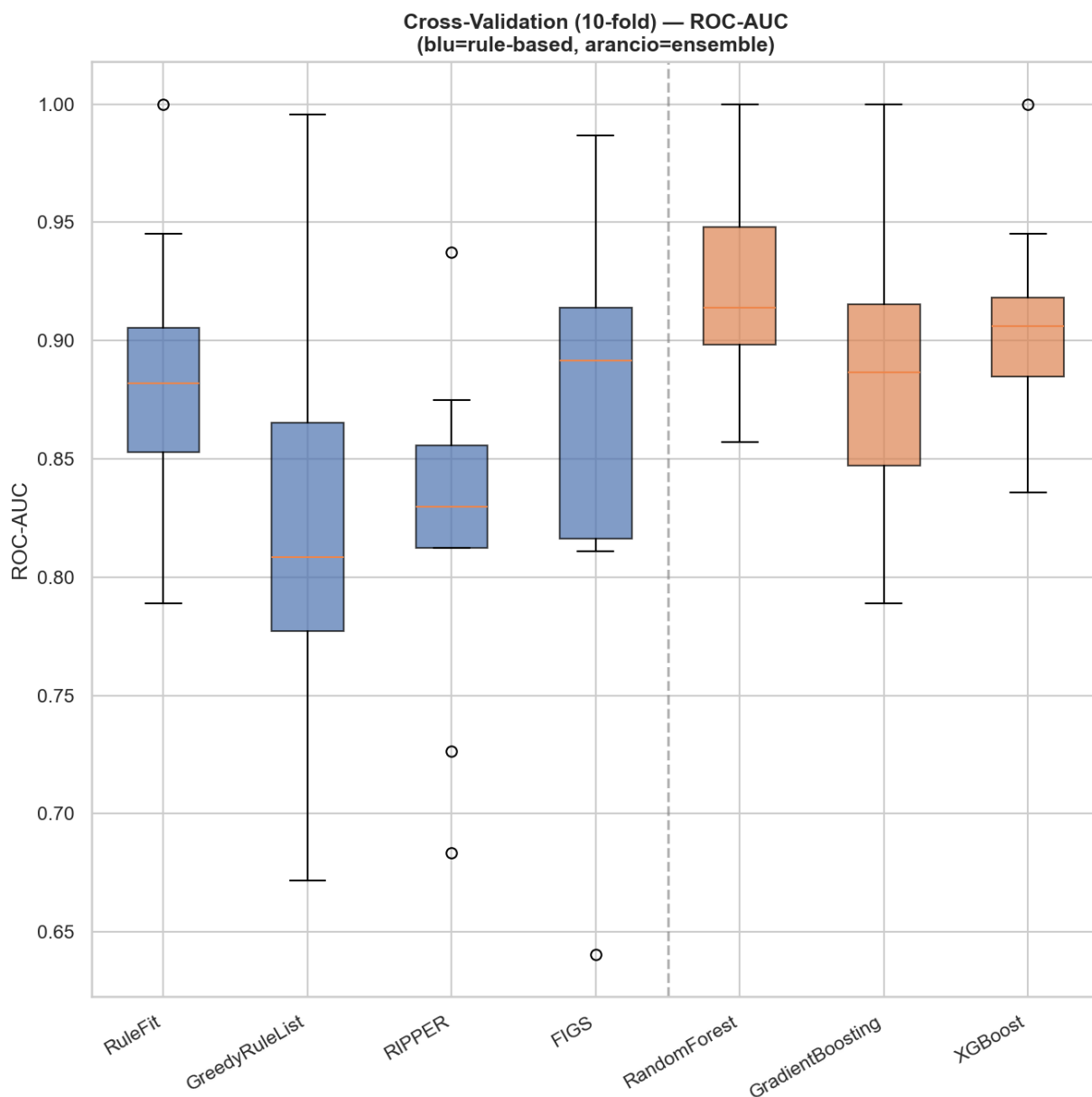
```
> -----
> FIGS-Fast Interpretable Greedy-Tree Sums:
> Predictions are made by summing the "Val" reached by traversing each tree.
> For classifiers, a softmax function is then applied to the sum.
> -----
time <= 73.500 (Tree #0 root)
  time <= 52.000 (split)
    Val: 0.077 0.923 (leaf)
```



```
    serum_sodium <= 137.000 (split)
      Val: 0.163 0.837 (leaf)
    ejection_fraction <= 30.000 (split)
      Val: 0.258 0.742 (leaf)
      Val: 0.926 0.074 (leaf)
  serum_creatinine <= 1.450 (split)
    Val: 0.898 0.102 (leaf)
  creatinine_phosphokinase <= 69.000 (split)
    Val: 1.059 -0.059 (leaf)
  time <= 104.000 (split)
    Val: 0.125 0.875 (leaf)
    serum_creatinine <= 1.965 (split)
      Val: 0.492 0.508 (leaf)
      Val: 1.258 -0.258 (leaf)

+
ejection_fraction <= 27.500 (Tree #1 root)
time <= 148.000 (split)
  Val: -0.104 0.104 (leaf)
  Val: -0.36 0.36 (leaf)
Val: 0.074 -0.074 (leaf)
```

3.5 Cross-Validation 10-fold



Il boxplot mostra la distribuzione del ROC-AUC su 10 fold stratificati del set trainval (≈ 240 campioni): mediana, IQR (bordi del box) e range (baffi). La stratificazione garantisce che ogni fold mantenga la stessa proporzione di deceduti ($\sim 32\%$). I modelli usano gli iperparametri ottimizzati sul validation set. Random Forest ha la mediana più alta e IQR contenuto: performance stabile e consistente. RuleFit è il miglior rule-based in CV. FIGS mostra alta varianza — instabile su dataset piccoli. GreedyRuleList usa un custom scorer che aggira un'incompatibilità con l'API sklearn.

3.6 RF, GRL e BRL senza feature time

Motivazione. La feature `time` (durata del follow-up) è la più predittiva del dataset. In contesti reali di triage questa informazione non è sempre disponibile al momento della prima valutazione. Questo esperimento valuta RF, GRL e BRL privandoli di `time`, forzandoli ad appoggiarsi ai segnali biochimici e demografici rimanenti. Tutti e tre i modelli vengono ri-tunati sul validation set (senza `time`) per garantire un confronto equo. Per BRL l'esperimento verifica inoltre se, in assenza della feature dominante, il MCMC produce antecedenti multi-condizionali.

Modello	Accuracy	F1	ROC-AUC
RandomForest	0.8667	0.7500	0.8960
RF_no_time	0.7500	0.5161	0.7754
GreedyRuleList	0.8333	0.6667	0.8331
GRL_no_time	0.6833	0.5366	0.7003
BayesianRuleList	0.7500	0.6809	0.8408
BRL_no_time	0.3167	0.4810	0.6528

Rimuovendo `time`, il ROC-AUC scende per tutti i modelli. RF e GRL subiscono un calo di circa 0.10–0.13 ma rimangono utilizzabili, appoggiandosi a `serum_creatinine` ed `ejection_fraction` — feature biochimiche disponibili alla prima valutazione. BRL soffre maggiormente (ROC-AUC scende a 0.65, accuracy al 32%): senza `time`, il modello tende a classificare la maggior parte dei campioni come deceduti, riflettendo la difficoltà del MCMC nel bilanciare le soglie senza il segnale dominante.

Regole GRL senza time. Con la rimozione di `time`, GreedyRuleList apprende dai segnali biochimici: `serum_creatinine` ed `ejection_fraction` diventano le feature dominanti.

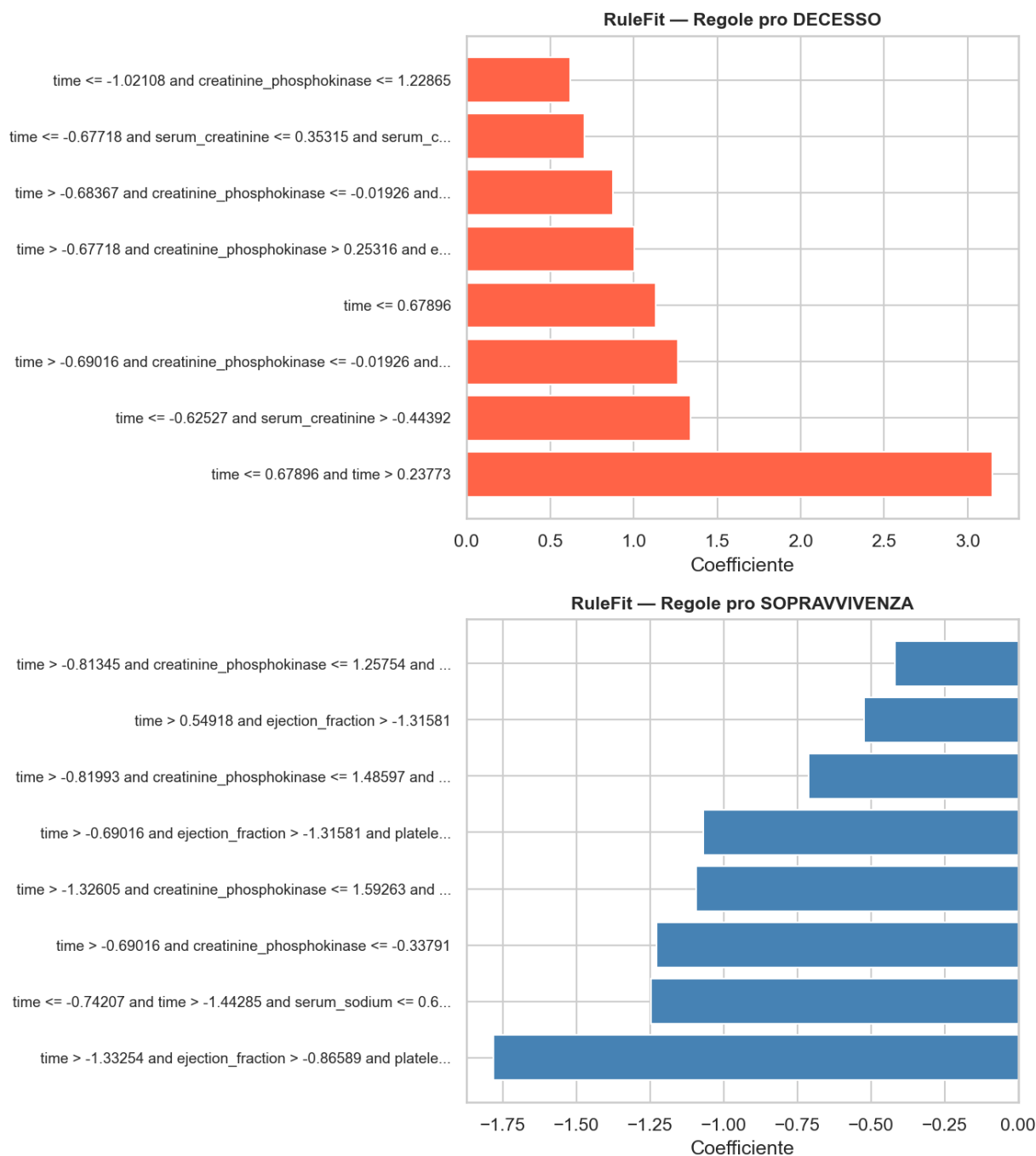
Condizione	Precisione	Recall	Predizione	Campioni
<code>serum_creatinine > 1.450</code>	0.658	0.431	Decesso	38
<code>ejection_fraction <= 27.500</code>	0.615	0.276	Decesso	26
<code>creatinine_phosphokinase > 5211.000</code>	1.000	0.034	Decesso	2
<code>age > 81.500</code>	0.500	0.025	Sopravvissuto	6
<code>diabetes > 0.500</code>	0.816	0.331	Sopravvissuto	49
<code>serum_creatinine <= 0.650</code>	0.667	0.017	Sopravvissuto	3
DEFAULT	0.964	0.438	Sopravvissuto	55

Regole BRL senza time. Rimuovendo `time`, BayesianRuleList continua a produrre antecedenti a condizione singola: `ejection_fraction` e `serum_creatinine` vengono usate in regole sequenziali separate, non congiunte. La preferenza per antecedenti unari è quindi una proprietà intrinseca del MCMC su questo dataset, indipendente dalla dominanza di `time`.

Condizione	P(decesso)	IC 95%
<code>ejection_fraction ∈ (15.00, 30.00]</code>	61.9%	46.9%–75.8%
<code>serum_creatinine ∈ (1.40, 6.80]</code>	53.1%	36.0%–69.8%
DEFAULT (nessuna regola attivata)	16.2%	10.0%–23.6%

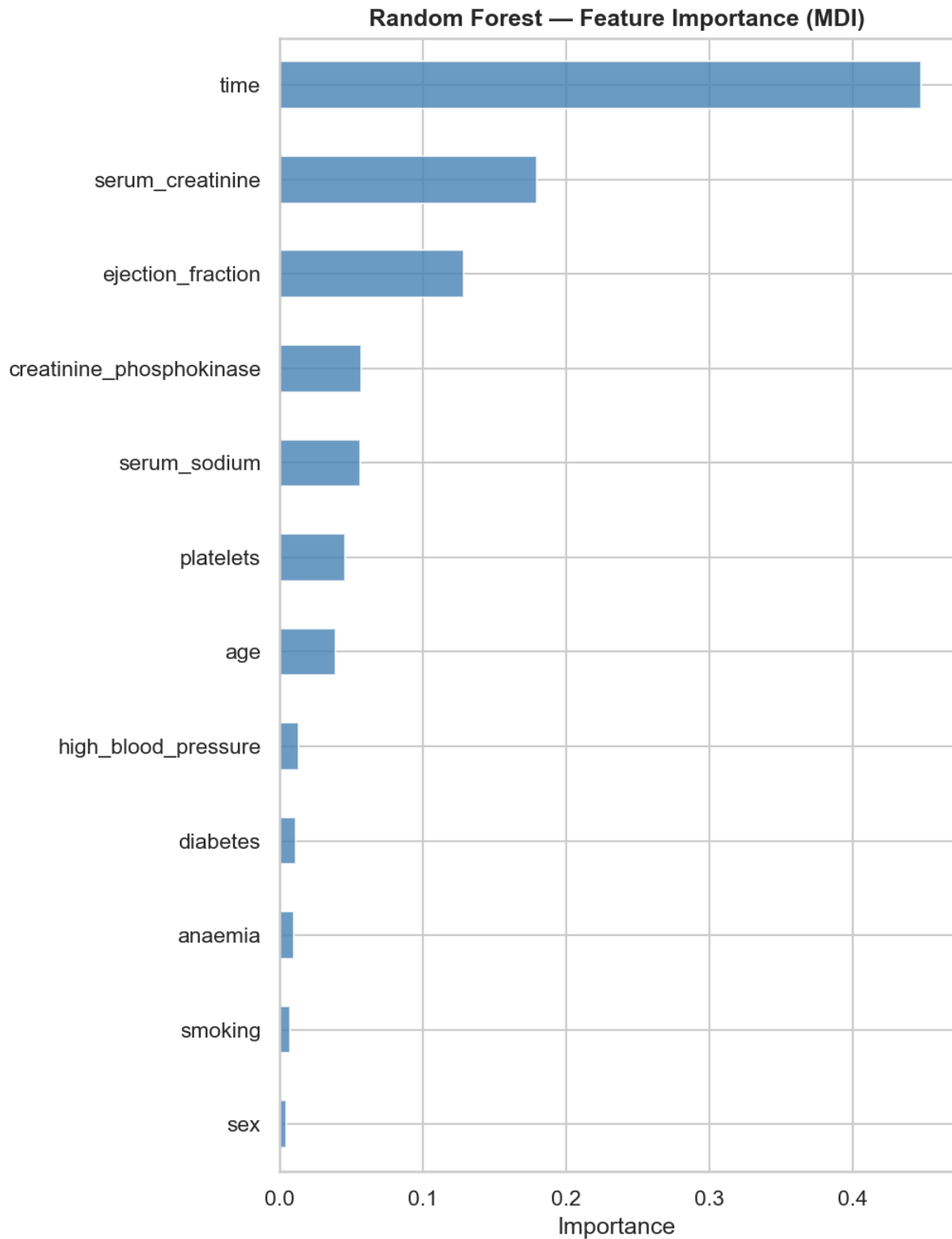
3.7 Analisi delle Feature

Regole RuleFit — coefficienti Lasso.



Coefficienti Lasso delle regole estratte da RuleFit: positivo (barre rosse) $\Rightarrow$ la regola aumenta il rischio di decesso; negativo (blu) $\Rightarrow$ lo riduce. Le regole pro-decesso con peso più alto coinvolgono **time** basso e **serum_creatinine** alta. Le regole pro-sopravvivenza implicano **ejection_fraction** più alta e follow-up lungo. La penalizzazione Lasso azzerava i coefficienti ridondanti, mantenendo solo le regole con effetto reale sulla predizione.

Feature Importance — Random Forest (MDI).



L'importanza MDI misura di quanto ogni feature riduce l'impurità media negli alberi del Random Forest ($n_estimators=100$): `time` domina nettamente, seguita da `serum_creatinine` ed `ejection_fraction`. Questa gerarchia è confermata in modo indipendente da RuleFit (sezione 3.4): le regole con coefficiente Lasso più alto coinvolgono `time` basso e `serum_creatinine` alta. Due metodi radicalmente diversi — importance da ensemble di alberi vs regressione Lasso su regole es-

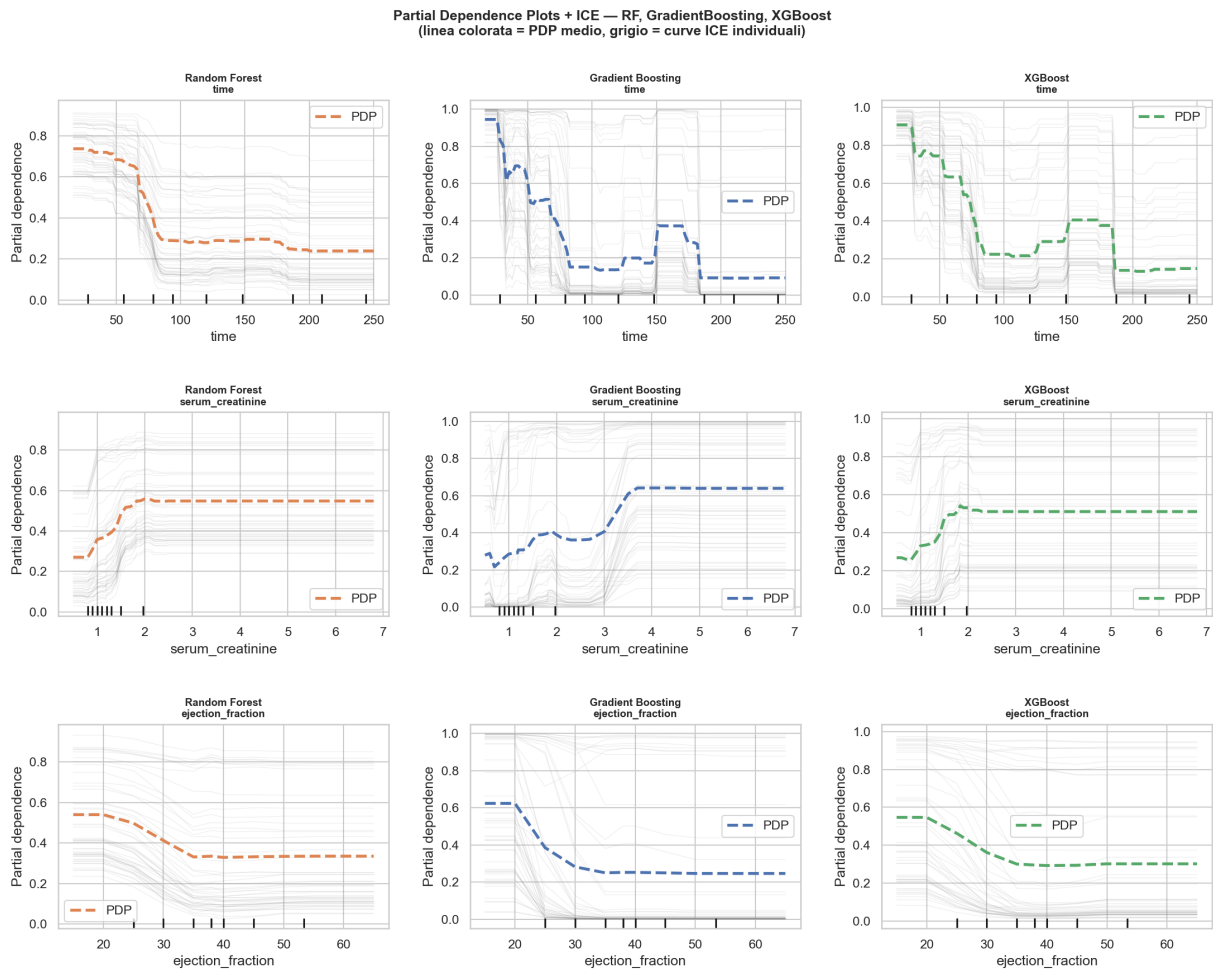
tratte — convergono sugli stessi tre segnali fisiopatologici, coerenti con la letteratura sull'insufficienza cardiaca.

3.8 Partial Dependence Plots e ICE

I Partial Dependence Plot (PDP) e le Individual Conditional Expectation (ICE) sono strumenti post-hoc per visualizzare l'effetto marginale di una feature sulla predizione, mantenendo le altre feature ai loro valori osservati (marginalizzazione).

Motivazione: perché sui modelli ensemble. PDP e ICE sono strumenti nati per spiegare modelli black-box: la loro utilità è massima sui modelli ensemble (RF, GradientBoosting, XGBoost) la cui logica interna non è direttamente ispezionabile. Per i modelli rule-based (RuleFit, GRL, BRL, FIGS, SkopeRules, RIPPER) le regole apprese (o i coefficienti Lasso nel caso di RuleFit) sono già la spiegazione completa del comportamento del modello, quindi un PDP sarebbe ridondante. L'analisi che segue si concentra dunque sui tre ensemble, verificando se modelli con bias induttivi diversi (bagging vs boosting) apprendono pattern simili.

PDP 1D + ICE — tre feature principali. Ogni cella mostra la curva PDP media (linea colorata) sovrapposta alle curve ICE individuali (grigio, 50 campioni). Le tre feature analizzate — `time`, `serum_creatinine` ed `ejection_fraction` — sono le più informative secondo la feature importance MDI. I tre modelli confrontati sono Random Forest (arancio), Gradient Boosting (blu) e XGBoost (verde).



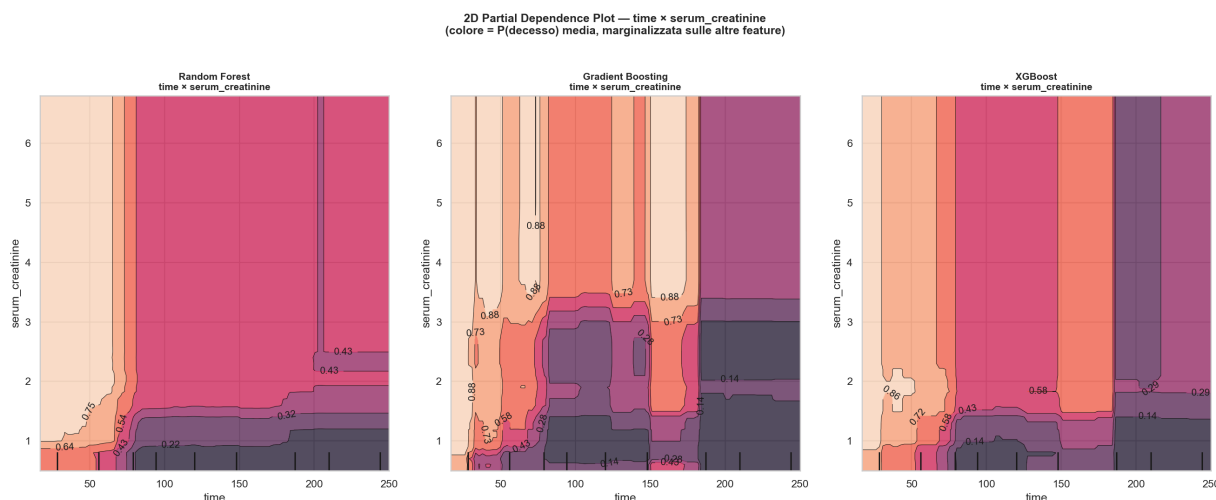
Lettura dei risultati. Tutti e tre gli ensemble apprendono pattern coerenti: `time` ha relazione fortemente decrescente con soglia critica intorno a 70–80 giorni; `serum_creatinine` ha effetto

crescente (alta creatinina = alto rischio renale); `ejection_fraction` ha effetto decrescente. La convergenza su questi pattern — nonostante RF usi bagging e GB/XGBoost usino boosting — è una validazione qualitativa robusta: i tre modelli, partendo da bias induttivi diversi, identificano le stesse relazioni fisiopatologiche.

Differenze qualitative tra i modelli. RF produce curve più lisce e monotone, conseguenza del bagging che media molte predizioni indipendenti. Gradient Boosting e XGBoost, costruendo alberi sequenzialmente sui residui, mostrano transizioni più nette e talvolta non-monotonie locali (es. piccoli rimbalzi su `time` attorno a 150–180 giorni), riflettendo la maggiore capacità del boosting di modellare pattern fine. Le curve ICE rivelano che per `time` la maggior parte dei campioni segue il trend medio (eterogeneità contenuta), mentre per `serum_creatinine` alcune ICE divergono in particolare nei modelli boosting — segnale di interazioni con altre feature catturate dal modello.

PDP 2D — interazione `time` × `serum_creatinine`. Il PDP bidimensionale mostra la probabilità di decesso media in funzione congiunta di `time` e `serum_creatinine`, marginalizzando sulle restanti feature. Non esiste un analogo ICE per il caso 2D (richiederebbe una superficie per ogni campione).

Motivazione della scelta della coppia. La coppia `time` × `serum_creatinine` è stata selezionata in base a due criteri: (i) sono le due feature più informative secondo la feature importance MDI, con `serum_creatinine` che ha la correlazione più alta con `DEATH_EVENT` tra le feature biochimiche (+0.29, contro −0.27 di `ejection_fraction`); (ii) compaiono congiuntamente nella maggior parte delle SkopeRules apprese (es. `time` ≤ 73.5 AND `serum_creatinine` > 0.85) e nelle regole RuleFit con coefficiente Lasso più alto — segnale che il modello stesso le considera interagenti, e quindi la coppia più candidata a esibire una vera interazione non-additiva.



Lettura dei risultati. L'angolo in alto a sinistra (follow-up breve e creatinina alta) corrisponde al massimo rischio in tutti e tre i modelli ($P(\text{decesso}) \approx 0.6\text{--}0.88$). L'angolo in basso a destra (follow-up lungo e creatinina bassa) corrisponde alla minima probabilità di decesso ($P(\text{decesso}) \approx 0.14\text{--}0.22$). La regione di transizione è governata principalmente da `time`: la soglia critica appare intorno a 70–80 giorni in modo abbastanza indipendente dal valore di `serum_creatinine`, coerente con la dominanza di `time` osservata nella feature importance.

Confronto tra modelli. Random Forest mostra curve di livello smooth e prevalentemente verticali: l'effetto dominante è `time`, con `serum_creatinine` che modula gradualmente il rischio soprattutto a follow-up brevi. Gradient Boosting evidenzia un pattern più complesso, con "isole" di rischio elevato ($P \approx 0.88$) e regioni di basso rischio anche per follow-up intermedi: il boosting cattura una vera non-additività tra le due feature. XGBoost si colloca tra i due, con transizioni nette ma più frammentate rispetto a RF. La convergenza qualitativa dei tre modelli (gradiente diagonale

dall'angolo alto-sinistro a quello basso-destro) conferma che la struttura del rischio è solida; le differenze locali riflettono il bias induttivo di ciascun algoritmo: bagging → smoothing, boosting → pattern fine.

4 Conclusioni

Confronto tra approcci. I modelli ensemble raggiungono ROC-AUC superiore (Random Forest 0.896, XGBoost 0.843) ma il divario con i rule-based non è netto: GreedyRuleList (0.833) è competitivo su un dataset di soli 299 campioni, dove la semplicità strutturale riduce il rischio di overfitting. BayesianRuleList ottiene il miglior F1 tra i rule-based (0.681) e produce stime probabilistiche con IC al 95%, utili per comunicare l'incertezza al clinico. GreedyRuleList offre il secondo F1 rule-based (0.667) con una lista ordinata di condizioni particolarmente leggibile da un clinico. RuleFit assegna coefficienti espliciti a ogni regola, rendendo visibile il peso relativo di ciascuna. FIGS ha struttura additiva particolarmente trasparente. SkopeRules raggiunge Precision=1.0 con Recall molto basso: non produce mai falsi positivi, ma identifica solo una piccola frazione dei deceduti — utile quando ogni allarme deve corrispondere a un caso critico. RIPPER apprende regole congiuntive (AND tra più condizioni) in modo incrementale con potatura: nonostante la maggiore espressività strutturale rispetto a GRL, ottiene il ROC-AUC più basso di tutti i modelli. Il vantaggio strutturale non si traduce in migliore capacità discriminativa su questo dataset.

Tuning degli iperparametri. La selezione sul validation set ha mostrato che per BayesianRuleList la cardinalità ottimale è 3 (sebbene in pratica il MCMC converga su regole a condizione singola, come discusso in sezione 3.4), mentre per SkopeRules `max_depth=2` con soglie `precision_min=0.6` e `recall_min=0.3` produce regole di qualità senza eccessiva proliferazione. Per gli ensemble, i valori ottimali tendono a configurazioni più semplici (meno stimatori, alberi poco profondi), coerente con la dimensione ridotta del dataset.

Precision vs Recall in ambito clinico. Il trade-off tra precisione e recall non è neutro in contesti medici. Alta recall minimizza i falsi negativi (morti non rilevate), che in clinica sono più pericolosi dei falsi positivi (falsi allarmi). Alta precisione garantisce invece che ogni segnalazione sia affidabile, preferibile quando si pianificano interventi invasivi. La scelta del modello dipende quindi dal contesto d'uso, e la trasparenza dei modelli rule-based è la condizione necessaria per fare questa scelta in modo consapevole.

Validazione qualitativa delle regole. Le feature più ricorrenti — `time` (follow-up), `serum_creatinine` ed `ejection_fraction` — sono coerenti con la letteratura clinica sull'insufficienza cardiaca: follow-up breve segnala una fase acuta critica, creatinina alta indica insufficienza renale, ejection fraction bassa riflette ridotta capacità di pompa. Il fatto che i modelli apprendano autonomamente queste relazioni costituisce una validazione qualitativa.

Ablazione della feature `time`. Come mostrato nella sezione 3.6, rimuovendo `time` il ROC-AUC scende per tutti e tre i modelli, confermando il suo peso dominante. RF e GRL rimangono utilizzabili in triage; BRL degrada significativamente, evidenziando una maggiore dipendenza dal segnale di follow-up.

Sintesi finale. L'analisi delle feature (sezione 3.7) e i Partial Dependence Plots (sezione 3.8) hanno mostrato che metodi radicalmente diversi — feature importance MDI, coefficienti Lasso su regole e PDP/ICE sugli ensemble — convergono sugli stessi tre segnali clinici dominanti: `time`, `serum_creatinine` ed `ejection_fraction`. Questa convergenza, unita alla coerenza con la letteratura sull'insufficienza cardiaca, è il risultato più solido del progetto.